Modern DevOps

Bridging the Gap Between Development and IT Operations

INTRODUCTION & BEST PRACTICES

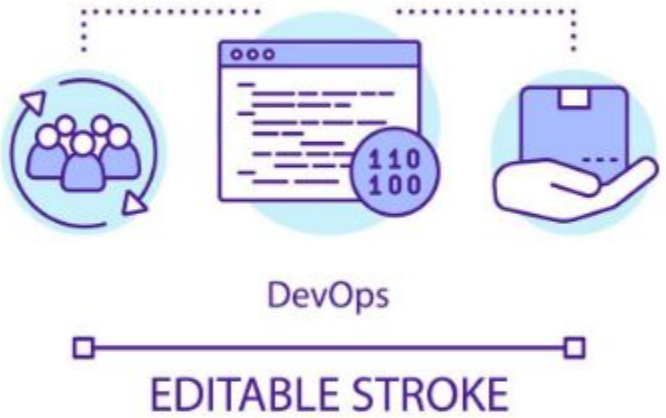
Defining DevOps

Unified Culture

DevOps is a set of practices that combines software development (**Dev**) and IT operations (**Ops**).

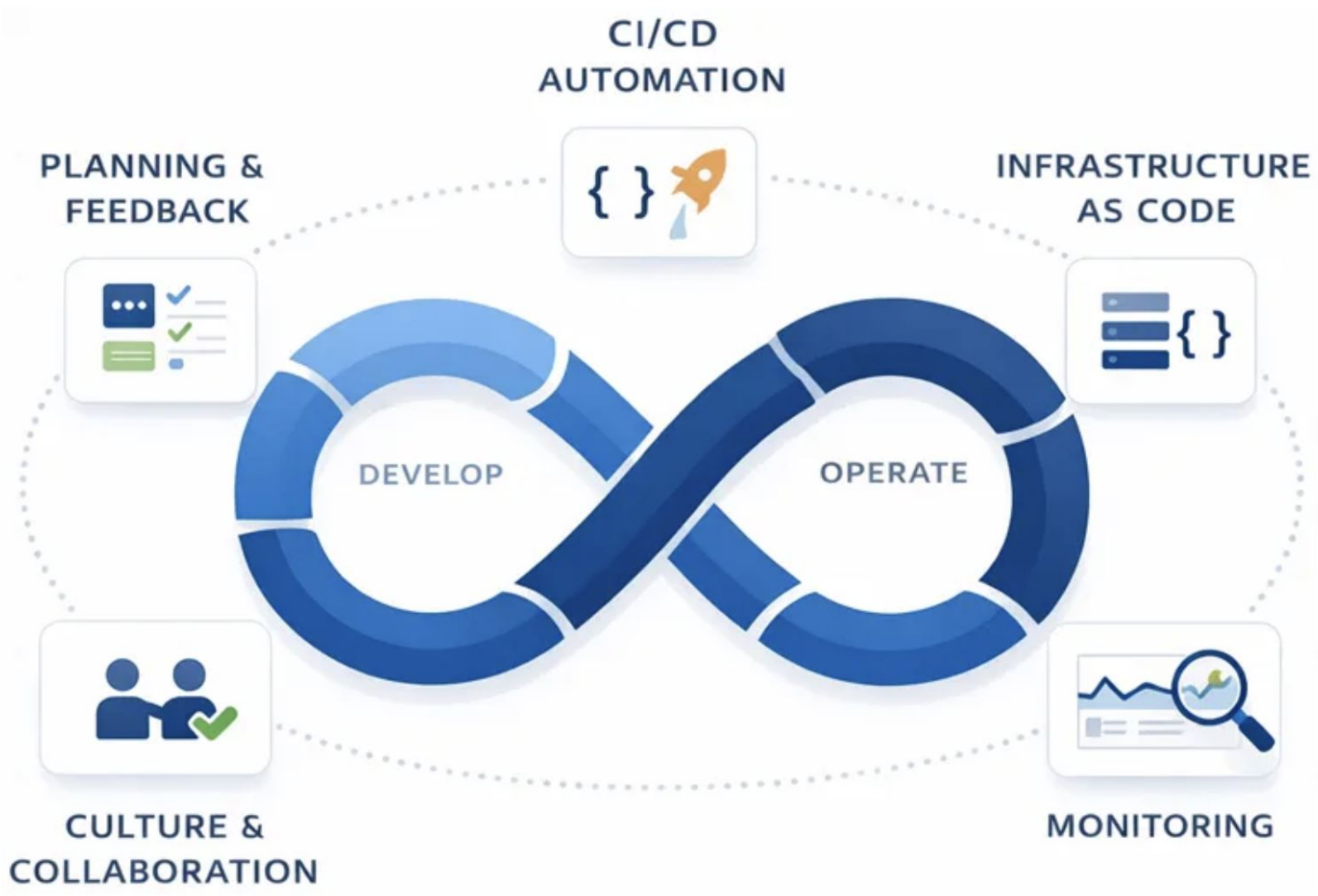
It aims to shorten the systems development life cycle and provide continuous delivery with high software quality.

Goal: Merge the "speed" of developers with the "stability" of operations teams.



CALMS framework

Culture • Automation • Lean • Measurement • Sharing



The DevOps Pipeline

CI

Continuous Integration: Merging code changes into a shared repo frequently to trigger automated builds.

CD

Continuous Delivery: Ensuring the code is always in a deployable state to a production-like environment.

CDp

Continuous Deployment: Automatically pushing every change that passes the pipeline into production.

CM

Continuous Monitoring: Real-time tracking of application performance and user behavior.

The DevOps SDLC

A Continuous, Circular Journey

1. Plan

Continuous Planning

DevOps planning happens in small, iterative chunks (Sprints) rather than massive upfront documents.

DevOps Twist: Heavy emphasis on "Shift Left"—discussing security and operations requirements during the initial phase.

2. Code (Develop)

Small Increments

Developers write code in manageable pieces and merge frequently into a shared repository.

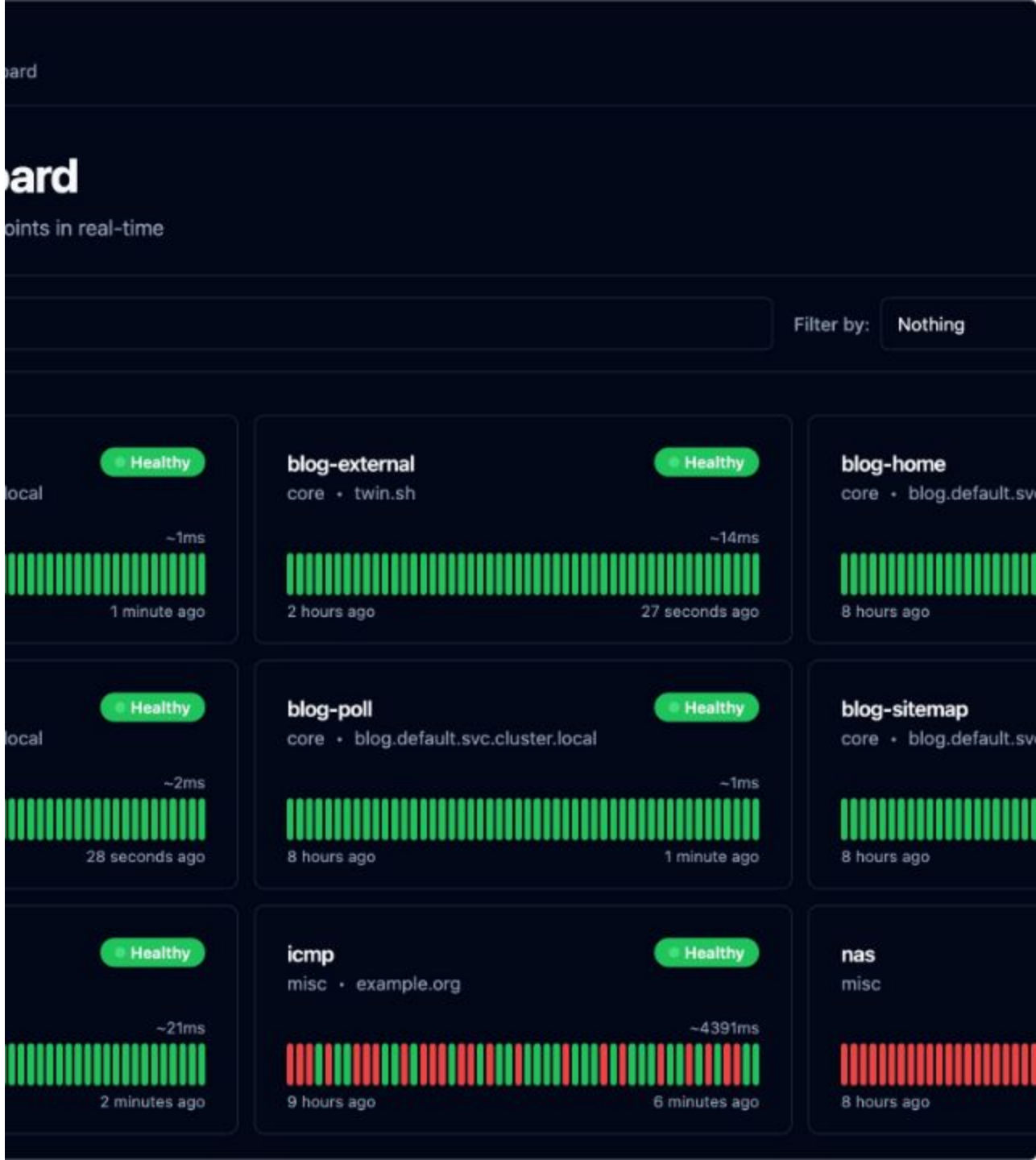
DevOps Twist: Version Control Systems (like Git) are non-negotiable. Merging occurs multiple times a day to prevent "integration hell."

3. Build & Test (CI)

The Heartbeat

As soon as code is committed, an automated "Build" is triggered to compile code and resolve dependencies.

DevOps Twist: This is the CI heart. If a build fails, the team is notified immediately, keeping the main branch stable.



4. Release (CD)

Boring is Good

Once tests pass, the code is packaged as an artifact (e.g., Docker container) and stored in a registry.

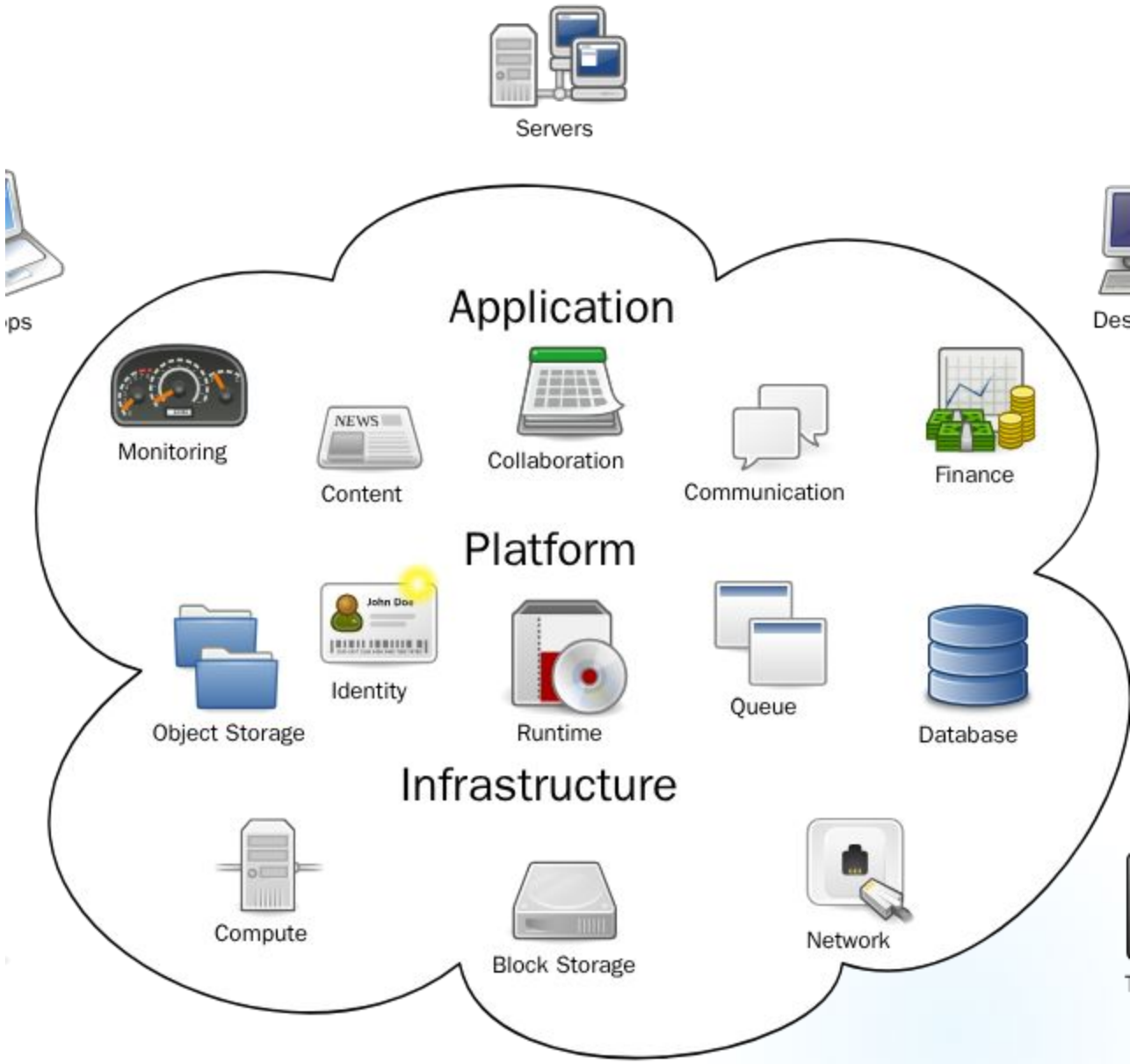
DevOps Twist: Releases should be predictable and "boring" because they've been automatically validated multiple times.

5. Deploy

Pushing to Production

Software is moved from staging to the live environment where actual users can access it.

DevOps Twist: Utilizing Infrastructure as Code (IaC) to ensure environments are identical every time, eliminating configuration drift.

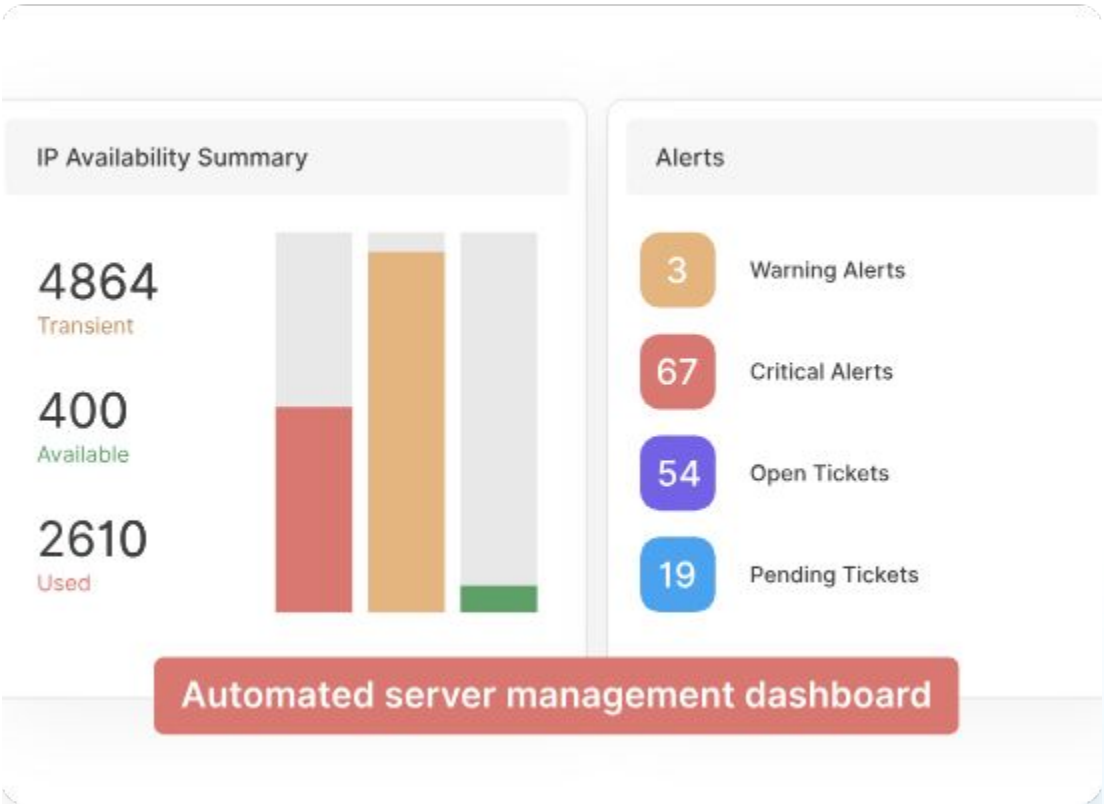


6. Operate

Maintenance & Scale

Maintaining software health, managing server resources, and ensuring high availability for global users.

DevOps Twist: Ops teams use automation for configurations and patching rather than manual server updates.

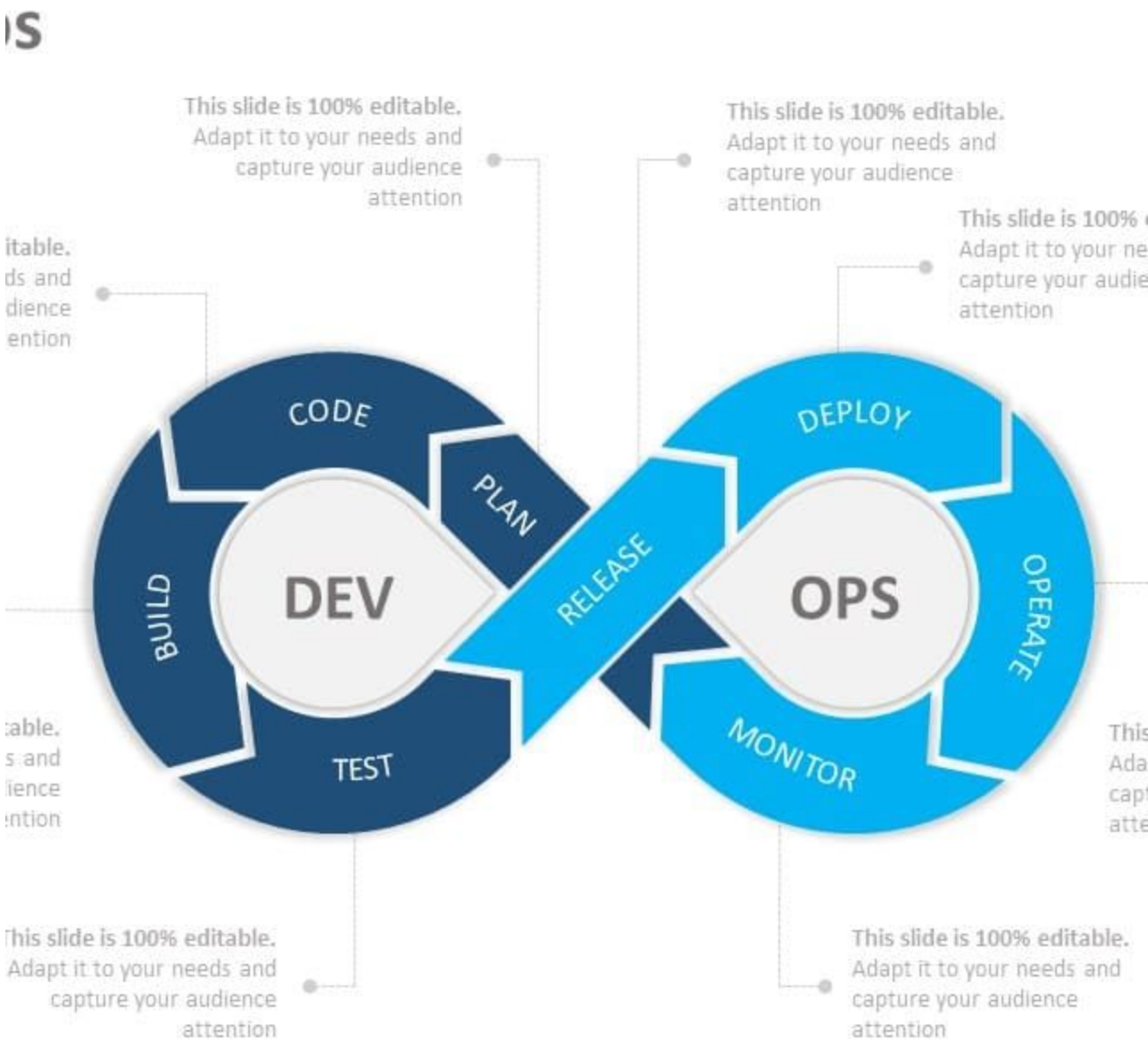


7. Monitor & Feedback

Closing the Loop

Data is collected on application performance, errors, latency, and user behavior in real-time.

DevOps Twist: This data provides a direct Feedback Loop to the Plan phase for the next iteration of the software.



Traditional vs. DevOps SDLC

Feature	Traditional (Waterfall)	DevOps SDLC
Structure	Linear / Sequential	Continuous / Circular
Release Frequency	Months or Years	Days or Hours
Feedback	At the very end	Constant and immediate
Testing	Separate phase after coding	Automated & Integrated throughout
Collaboration	Siloed (Dev vs. Ops)	Unified Shared Responsibility

Advanced Testing Strategies

Building Quality from the Foundation Up

The Testing Pyramid

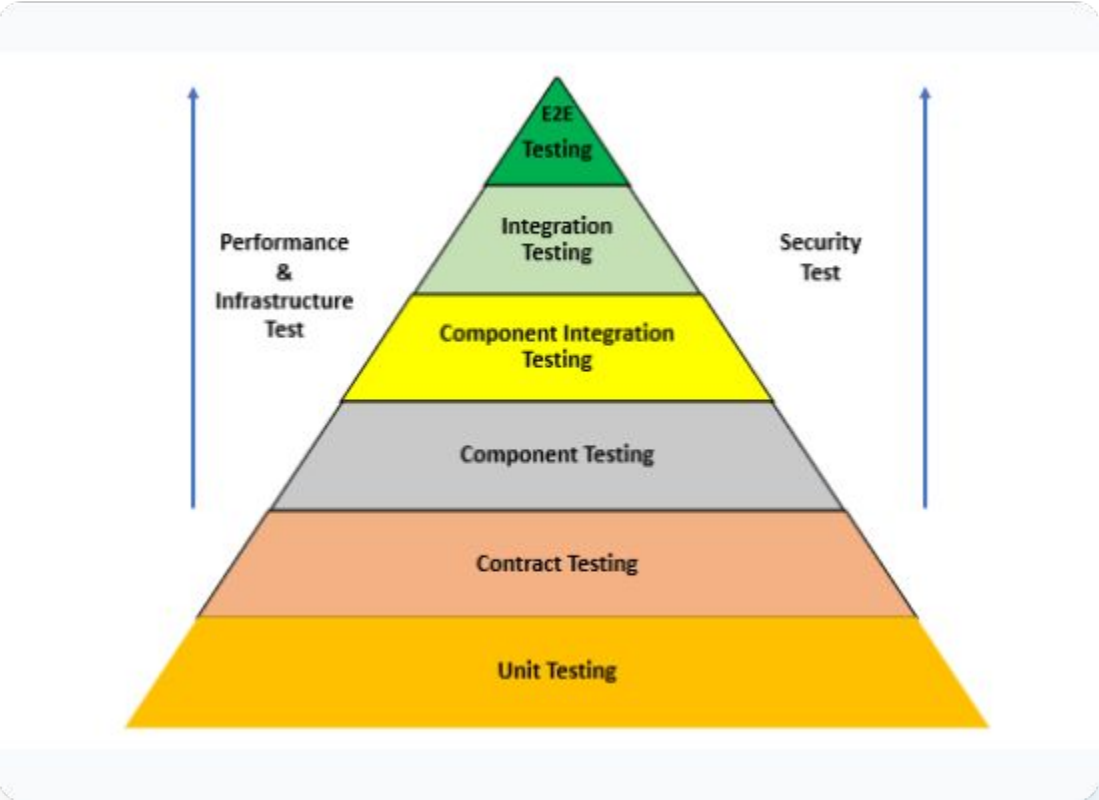
Hierarchy of Automation

The pyramid analogy suggests that a healthy testing suite should have a broad base of unit tests.

E2E (Top): Smallest amount. High cost, slow, fragile.

Integration (Middle): Moderate amount. Validates component interaction.

Unit (Base): Largest amount. Fast, reliable, cheap.



Types of Automated Tests

Unit Tests

Testing individual functions or methods in isolation. The fastest feedback loop for developers.

Integration

Verifying that different modules or services work together correctly (e.g., API to Database).

End-to-End

Simulating real user scenarios from start to finish across the entire system stack.

Clean Code Principles for Testing

SRP

Single Responsibility Principle: A module or class should have one, and only one, reason to change.

Testing becomes trivial because there are fewer dependencies to mock and scenarios to cover.

SoC

Separation of Concerns: Distinct sections should handle distinct functionalities.

By separating logic from UI or data access, we can test business rules without launching a heavy browser or database.

The Cost of Bad Code

The Testing Nightmare

Failing to adhere to SRP and SoC leads to "God Objects"—classes that do everything.

The Downside:

- Extremely fragile tests that break with any change.
- Impossible to mock dependencies effectively.
- Technical debt that slows down the entire CI/CD pipeline.

Why DevOps Matters: The Impact

208x

More Frequent Deploys

High Performance Results

According to DORA metrics, high-performing DevOps teams achieve significantly faster lead times and lower failure rates.

They recover from incidents **2,604x faster** than low performers, proving that speed and stability can co-exist.

Key Takeaways for Success

- **Automate Everything:** From builds to infrastructure, manual steps are the enemy of scale.
- **Shared Responsibility:** Dev and Ops are one team; everyone owns the software's uptime and quality.
- **Small Batches:** Deploy smaller changes more often to reduce risk and simplify troubleshooting.
- **Measure What Matters:** Use monitoring to drive your planning—data should always beat intuition.

Summary & Questions?